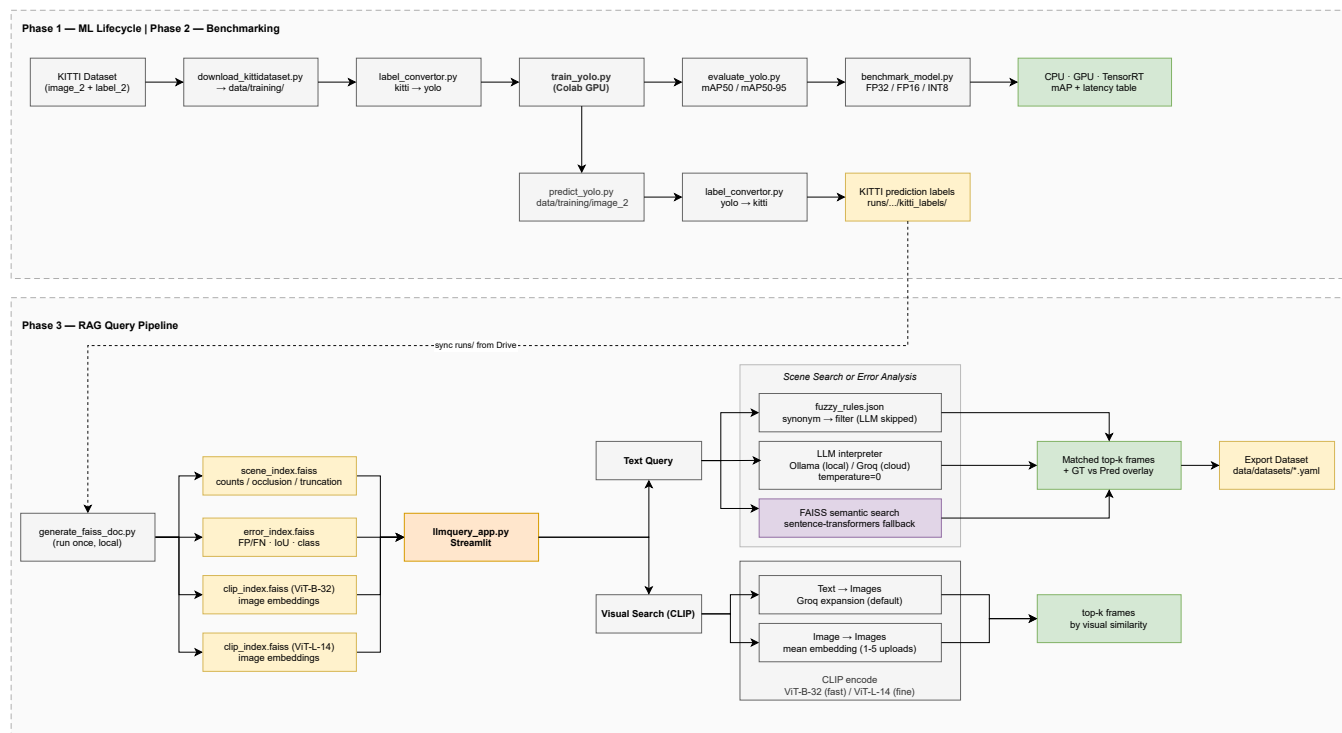


KITTI Object Detection: Training, Benchmarking and RAG Dataset Query

An end-to-end pipeline built on the KITTI autonomous driving dataset, covering three phases: training a YOLO model from raw sensor data, benchmarking it across deployment targets, and building a natural language query system for scenario mining and error analysis.



Repository Structure

```
preprocessing/download_kittidataset.py # dataset download
preprocessing/label_converter.py      # KITTI <-> YOLO label conversion
model/train_yolo.py                   # YOLOv8 training script
model/predict_yolo.py                 # run inference, save YOLO-format labels
model/evaluate_yolo.py               # mAP evaluation
model/benchmark_model.py              # FP32/FP16/INT8 benchmarking
utils/visualize_predictions.py        # GT vs prediction overlay
queries/generate_faiss_doc.py         # builds FAISS indexes
queries/llmquery_app.py               # Streamlit query app
data/fuzzy_rules.json                 # synonym + filter rules for fuzzy
matching                             #
data/                                # generated indexes, docs, configs
data/datasets/                       # exported YAML dataset manifests
google_collab_training.ipynb         # end-to-end Colab notebook
```

Requirements

```
pip install -r requirements.txt
```

Phase 1: ML Lifecycle

Downloads the KITTI dataset, converts labels between KITTI and YOLO formats, trains a YOLO model on Google Colab, and evaluates with mAP50/mAP50-95. The notebook [google_collab_training.ipynb](#) sequences all steps end-to-end on Colab GPU.

The pipeline is split between Colab (GPU steps) and local (everything else).

Colab — [google_collab_training.ipynb](#):

```
# fetches images + labels into data/training/
download_kittidataset.py

# converts labels, splits into train/val -> kitti_yolo/
label_convertor.py "kitti2yolo"

# train YOLO
train_yolo.py --model model/yolov8n.pt --data data/kitti.yaml --epochs 50 --imgsz
640 --batch 16 --device 0

# mAP50, mAP50-95 on val set (can also be triggered locally)
evaluate_yolo.py --model "runs/detect/train/weights/best.pt" --data
"data/kitti.yaml"

# FP32/FP16/INT8 across CPU, GPU, TensorRT
benchmark_model.py --model "runs/detect/train/weights/best.pt" --data
"data/kitti.yaml"
```

Local (run after syncing [runs/](#) from Google Drive):

```
# side-by-side GT vs prediction for one image
visualize_predictions.py

# saves YOLO-format labels to runs/detect/predict/labels/
predict_yolo.py --model "runs/detect/train/weights/best.pt" --source
data/training/image_2 --project runs/detect --name predict

# converts prediction labels to KITTI format
label_convertor.py "yolo2kitti"

# builds scene + error FAISS indexes and CLIP indexes
generate_faiss_doc.py
```

```
# Streamlit query app & export query results as yaml dataset
llmquery_app.py
```

Phase 2: Model Benchmarking

Benchmarks the trained model across CPU, GPU (PyTorch), and TensorRT at FP32, FP16, and INT8 precision.

Backend	Precision	mAP50	Latency (ms)
CPU	FP32	0.8648	140.42
CPU	INT8	0.8682	717.26
GPU	FP32	0.8648	14.29
GPU	FP16	0.8647	15.16
TensorRT	FP32	0.8682	0.39
TensorRT	FP16	0.8682	0.40
TensorRT	INT8	0.8682	0.40

TensorRT accuracy uses the ONNX model as a proxy. Latency is measured from the actual TensorRT engine.

Relevant script: [model/benchmark_model.py](#)

Phase 3: RAG Query Pipeline

A Streamlit app ([queries/llmquery_app.py](#)) for querying the KITTI dataset without writing filter code. It has two independent entry points: text-based query and visual CLIP query.

[queries/generate_faiss_doc.py](#) builds the indexes first: a scene-level FAISS index (object counts, occlusion, truncation per frame), an error-level FAISS index (FP/FN documents with IoU, class, bounding box), and a CLIP image index for visual search. Run this once before launching the app.

Query flow



```

|      L--> Error Analysis (FP/FN, IoU, class, occlusion)
|      |      |--> same LLM / fuzzy / semantic path
|      |      L--> results show GT vs prediction overlay
|
|      L--> Visual Search (CLIP)
|      |
|      |      +--> Text -> Images (describe what you want to find)
|      |      |      |--> query expansion via Groq (optional)
|      |      |      |--> CLIP text encode --> FAISS image index search
|      |      |      L--> ViT-B-32 (512-dim, fast) or ViT-L-14 (768-dim, finer)
|      |      |
|      |      L--> Image -> Images (upload 1-5 example images)
|      |      |      |--> CLIP image encode --> mean embedding
|      |      |      L--> ViT-B-32 or ViT-L-14 index search

```

Text query details

Both Scene Search and Error Analysis go through the same resolution path. First, the query is checked against `fuzzy_rules.json` terms like "crowded", "few cyclists", "heavy occlusion" are pre-mapped to exact numeric filters with synonyms. If the query is fully covered by fuzzy rules (after stripping stopwords with no filter intent), the LLM is skipped entirely and filters are applied directly. The UI shows "fuzzy rules only — LLM skipped" in this case.

If fuzzy rules don't cover everything, the query goes to the LLM. Two backends are supported and switchable at runtime from the sidebar: Ollama (local, private, no API key) and Groq (cloud, ~10x faster). Both run at temperature=0 for deterministic output. The LLM returns structured filters which are applied against the FAISS index. If nothing matches, it falls back to sentence-transformer semantic search over the same index.

Error Analysis results include a side-by-side overlay: GT boxes in green, predictions in yellow, false positives in red, false negatives in blue.

Dataset export

After any query returns results, an "Export Dataset (YAML)" button appears below the frames. Clicking it writes a manifest to `data/datasets/` that records which frames matched, the total match count, and the LLM filter output that produced them.

```

query: more than 5 pedestrians
mode: Scene Search
exported_at: '2024-06-30T14:22:01'
total_matched: 209
exported: 5
llm_output:
  filters:
    num_pedestrians:
      '>': 5
    semantic_query: scenes with many pedestrians
frames:
- id: '002445'

```

```
image: data/training/image_2/002445.png
label: data/training/label_2/002445.txt
```

This demonstrates the data lifecycle natural language query to a reproducible, traceable frame subset without coupling to a specific annotation tool.

Visual search details

Text-to-image search encodes a text description directly with CLIP and finds the most visually similar frames. When Groq is active, the query is first expanded into a richer visual description before encoding. This helps with short queries like "pedestrian crossing sign" where the raw text doesn't encode enough visual detail.

Image-to-image search takes one to five uploaded images, encodes each with CLIP, averages the embeddings, and retrieves the most similar frames. Multiple images make the query more robust.

Both modes support two CLIP models selectable from the sidebar. ViT-B-32 is faster and works well for scene-level queries. ViT-L-14 uses 14x14 pixel patches instead of 32x32, giving finer spatial resolution .> better for small objects like signs but slower.

Example queries

```
# Scene Search
"more than 5 pedestrians"
"busy intersection with few cyclists"      <- fuzzy fast path, LLM skipped
"heavy occlusion and more than 2 cars"

# Error Analysis
"false positives for cyclists"
"missed pedestrians with occlusion 3"
"IoU < 0.4 errors for cars"

# Visual Search (Text -> Images)
"street with parked vehicles"
"road with tram tracks"
"pedestrians crossing road"
"construction-cones on road"
```

Running the app

```
ollama run llama3      # skip if using Groq
cd queries
streamlit run .\llmquery_app.py
```

TensorRT requires the NVIDIA TensorRT SDK installed separately. For the query app, either Ollama running locally or a Groq API key (free at console.groq.com).

Querying Limitations — CLIP Visual Search

CLIP encodes the full frame as a single embedding vector. For scene-level queries (object counts, road type, occlusion level) this works well. For small or visual objects like traffic signs, construction workers, road markings. The object signal is diluted by dominant scene content (road, sky, vehicles) and retrieval degrades significantly.

Summary KPIs (10 queries, top-5 results):

Metric	ViT-L-14	ViT-B-32
Mean P@5 — all queries	0.52	0.56
Mean P@5 — scene-level queries	0.60	0.80
Mean P@5 — small object queries	0.40	0.20
Complete failure (P@5 = 0)	2 / 10	3 / 10
Hit Rate@5	0.80	0.80

Key findings:

- B-32 wins overall (Mean P@5 0.56 vs 0.52) and on scene-level queries (0.80 vs 0.60)
- L-14 wins on small/specific objects (0.40 vs 0.20)
- Both models fail completely on "construction workers" -> zero correct results in top-5 (lack of sufficient samples in training dataset)
- Examples for scene-level queries: railway tracks, parked vehicles, traffic lights, construction cones, vehicles at traffic light,...
- Examples for small object queries: pedestrian signs, speed signs,...
- Examples for complete failure queries: construction workers,...

Approaches tried:

- **Query expansion (Groq):** expands short queries into rich visual descriptions before CLIP encoding. Marginal improvement for text-to-image -> does not fix the fundamental full-frame granularity issue.
- **ViT-L-14 vs ViT-B-32:** finer patches improve small object recall marginally but full-frame encoding remains the bottleneck. However, both models switchable in the app sidebar.
- **Caption-based hybrid:** attempted with Salesforce/blip2-opt-2.7b -> did not produce meaningful captions on KITTI frames. Salesforce/instructblip-flan-t5-xl was too large to run locally.

Recommended path — two-stage cropping:

KITTI labels only cover Car, Pedestrian, and Cyclist but no signs, cones, or construction workers, which are exactly the queries that fail (P@5 = 0). The box source for Stage 2 therefore depends on the query category:

Query category	Box source for Stage 2
Car, Pedestrian, Cyclist	KITTI GT label files (already available)
Signs, cones, construction workers	OWL-ViT zero-shot prediction

1. **Stage 1:** full-frame CLIP FAISS search -> top-20 candidate frames (current implementation)

2. **Stage 1.5:** (*out-of-vocabulary queries only*) run [OWL-ViT](#) on each candidate frame with the query as text input -> OWL-ViT predicts bounding boxes zero-shot. Frames with no detected box are dropped. Since only 20 frames are processed, compute cost is negligible.
3. **Stage 2:** crop each region (GT box or OWL-ViT box), re-encode the crop with CLIP, re-rank candidates by crop-level similarity score.

This hybrid avoids the need for a separate region proposal network for standard KITTI categories while extending coverage to rare out-of-vocabulary targets via zero-shot detection. See [RegionCLIP](#) (Microsoft, CVPR 2022) and [OWL-ViT](#) (Google, NeurIPS 2022) for the learned equivalents of this pipeline, and [SearchAD](#) (Mercedes-Benz / Esslingen, CVPR 2026): a benchmark across 423k AD frames including KITTI, which confirms that spatial visual feature alignment outperforms full-frame CLIP for rare object retrieval.

Next Steps

- **FastAPI backend:** decouple query logic from Streamlit; FastAPI handles FAISS search, LLM routing, and CLIP encoding with multi-worker concurrency. Streamlit becomes a thin UI layer, enabling multiple simultaneous users.
- **Qdrant:** replace FAISS for named vectors and pre-filtered HNSW; stores text and CLIP embeddings per frame in a single collection, enabling filtered vector search without post-filtering accuracy loss.
- **RAG evaluation:** define ground-truth query→frame pairs; measure Precision@5, Recall@5, and per-query latency to benchmark retrieval quality objectively.
- **Two-stage CLIP retrieval:** region-crop re-ranking (see Querying Limitations) to improve small object recall for signs, construction workers, and sparse targets.
- **FiftyOne:** dataset curation, label quality inspection, and structured export to annotation pipelines.